

From GAN to WGAN

This PDF conversion (v1) was generated on October 10, 2025*

Online version: <https://lilianweng.github.io/posts/2017-08-20-gan/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2017-08-20-gan/>.

Contents

1	From GAN to WGAN	3
2	Kullback–Leibler and Jensen–Shannon Divergence#	4
3	Generative Adversarial Network (GAN)#	4
3.1	What is the optimal value for D ?#	5
3.2	What is the global optimal?#	6
3.3	What does the loss function represent?#	6
4	Problems in GANs#	7
4.1	Hard to achieve Nash equilibrium#	7
4.2	Low dimensional supports#	7
4.3	Vanishing gradient#	8
4.4	Mode collapse#	9
4.5	Lack of a proper evaluation metric#	9
5	Improved GAN Training#	9
6	Wasserstein GAN (WGAN)#	10
6.1	What is Wasserstein distance?#	10
6.2	Why Wasserstein is better than JS or KL divergence?#	11
6.3	Use Wasserstein distance as GAN loss function#	12
7	Example: Create New Pokemons!#	14
8	References#	14
9	Citation	15
10	References	16

[Lil'Log](#)

- [|](#)
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 From GAN to WGAN

Date: August 20, 2017 | Estimated Reading Time: 21 min | Author: Lilian Weng
Table of Contents

- [Kullback–Leibler and Jensen–Shannon Divergence](#)
- [Generative Adversarial Network \(GAN\)](#)
 - [What is the optimal value for D?](#)
 - [What is the global optimal?](#)
 - [What does the loss function represent?](#)
- [Problems in GANs](#)
 - [Hard to achieve Nash equilibrium](#)
 - [Low dimensional supports](#)
 - [Vanishing gradient](#)
 - [Mode collapse](#)
 - [Lack of a proper evaluation metric](#)
- [Improved GAN Training](#)
- [Wasserstein GAN \(WGAN\)](#)
 - [What is Wasserstein distance?](#)
 - [Why Wasserstein is better than JS or KL divergence?](#)
 - [Use Wasserstein distance as GAN loss function](#)
- [Example: Create New Pokemons!](#)
- [References](#)

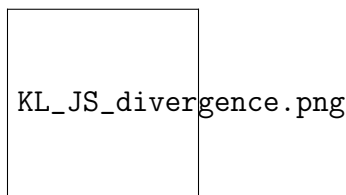


Figure 1: Given two Gaussian distribution, p with mean=0 and std=1 and q with mean=1 and std=1. The average of two distributions is labelled as $m=(p+q)/2$. KL divergence D_{KL} is asymmetric but JS divergence D_{JS} is symmetric.

[Updated on 2018-09-30: thanks to Yoonju, we have this post translated in [Korean!](#)
 [Updated on 2019-04-18: this post is also available on [arXiv](#).]

Generative adversarial network (GAN) has shown great results in many generative tasks to replicate the real-world rich content such as images, human language, and music. It is inspired by game theory: two models, a generator and a critic, are competing with each other while making each other stronger at the same time. However, it is rather challenging to train a GAN model, as people are facing issues like training instability or failure to converge.

Here I would like to explain the maths behind the generative adversarial network framework, why it is hard to be trained, and finally introduce a modified version of GAN intended to solve the training difficulties.

2 Kullback–Leibler and Jensen–Shannon Divergence#

Before we start examining GANs closely, let us first review two metrics for quantifying the similarity between two probability distributions.

(1) **KL (Kullback–Leibler) divergence** measures how one probability distribution p diverges from a second expected probability distribution q .

$$D_{\text{KL}}(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} dx$$

D_{KL} achieves the minimum zero when $p(x) = q(x)$ everywhere.

It is noticeable according to the formula that KL divergence is asymmetric. In cases where $p(x)$ is close to zero, but $q(x)$ is significantly non-zero, the q 's effect is disregarded. It could cause buggy results when we just want to measure the similarity between two equally important distributions.

(2) **Jensen–Shannon Divergence** is another measure of similarity between two probability distributions, bounded by $[0, 1]$. JS divergence is symmetric (yay!) and more smooth. Check this [Quora post](#) if you are interested in reading more about the comparison between KL divergence and JS divergence.

$$D_{\text{JS}}(p \parallel q) = \frac{1}{2} D_{\text{KL}}(p \parallel \frac{p+q}{2}) + \frac{1}{2} D_{\text{KL}}(q \parallel \frac{p+q}{2})$$

Some believe ([Huszar, 2015](#)) that one reason behind GANs' big success is switching the loss function from asymmetric KL divergence in traditional maximum-likelihood approach to symmetric JS divergence. We will discuss more on this point in the next section.

3 Generative Adversarial Network (GAN)#

GAN consists of two models:

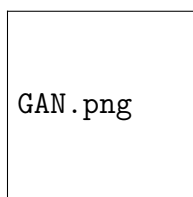


Figure 2: Architecture of a generative adversarial network. (Image source: www.kdnuggets.com/2017/01/generative-...-learning.html)

- A discriminator D estimates the probability of a given sample coming from the real dataset. It works as a critic and is optimized to tell the fake samples from the real ones.
- A generator G outputs synthetic samples given a noise variable input z (z brings in potential output diversity). It is trained to capture the real data distribution so that its generative samples can be as real as possible, or in other words, can trick the discriminator to offer a high probability.

These two models compete against each other during the training process: the generator G is trying hard to trick the discriminator, while the critic model D is trying hard not to be cheated. This interesting zero-sum game between two models motivates both to improve their functionalities.

Given,

Symbol Meaning Notes

p_z Data distribution over noise input z Usually, just uniform.

p_g The generator's distribution over data x

p_r Data distribution over real sample x

On one hand, we want to make sure the discriminator D 's decisions over real data are accurate by maximizing $\mathbb{E}_{x \sim p_r(x)} [\log D(x)]$. Meanwhile, given a fake sample $G(z)$, $z \sim p_z(z)$, the discriminator is expected to output a probability, $D(G(z))$, close to zero by maximizing $\mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]$.

On the other hand, the generator is trained to increase the chances of D producing a high probability for a fake example, thus to minimize $\mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]$.

When combining both aspects together, D and G are playing a **minimax game** in which we should optimize the following loss function:

$$\begin{aligned} \min_G \max_D L(D, G) &= \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \\ &\quad \&= \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{x \sim p_g(x)} [\log(1 - D(x))] \end{aligned}$$

($\mathbb{E}_{x \sim p_r(x)} [\log D(x)]$ has no impact on G during gradient descent updates.)

3.1 What is the optimal value for D ?#

Now we have a well-defined loss function. Let's first examine what is the best value for D .

$$L(G, D) = \int_x \big(p_r(x) \log(D(x)) + p_g(x) \log(1 - D(x)) \big) dx$$

Since we are interested in what is the best value of $D(x)$ to maximize $L(G, D)$, let us label

$$\tilde{x} = D(x), A = p_r(x), B = p_g(x)$$

And then what is inside the integral (we can safely ignore the integral because x is sampled over all the possible values) is:

$$\begin{aligned} f(\tilde{x}) &= A \log \tilde{x} + B \log(1 - \tilde{x}) \\ f'(\tilde{x}) &= A \frac{1}{\tilde{x}} - B \frac{1}{1 - \tilde{x}} \\ &= \frac{1}{\tilde{x}(1 - \tilde{x})} (A(1 - \tilde{x}) - B\tilde{x}) \end{aligned}$$

Thus, set $f'(\tilde{x}) = 0$, we get the best value of the discriminator: $D^*(x) = \tilde{x}^* = \frac{A}{A+B} = \frac{p_r(x)}{p_r(x) + p_g(x)}$ in $[0, 1]$.

Once the generator is trained to its optimal, p_g gets very close to p_r . When $p_g = p_r$, $D^*(x)$ becomes $1/2$.

3.2 What is the global optimal?#

When both G and D are at their optimal values, we have $p_g = p_r$ and $D^*(x) = 1/2$ and the loss function becomes:

$$L(G, D^*) = \int_x \big(p_r(x) \log(D^*(x)) + p_g(x) \log(1 - D^*(x)) \big) dx = \int_x p_r(x) \log \frac{1}{2} dx + \int_x p_g(x) \log \frac{1}{2} dx = -2 \log 2$$

3.3 What does the loss function represent?#

According to the formula listed in the [previous section](#), JS divergence between p_r and p_g can be computed as:

$$\begin{aligned} D_{JS}(p_r || p_g) &= \frac{1}{2} D_{KL}(p_r || \frac{p_r + p_g}{2}) + \frac{1}{2} D_{KL}(p_g || \frac{p_r + p_g}{2}) \\ &= \frac{1}{2} \big(\log 2 + \int_x p_r(x) \log \frac{p_r(x)}{p_r(x) + p_g(x)} dx \big) + \frac{1}{2} \big(\log 2 + \int_x p_g(x) \log \frac{p_g(x)}{p_r(x) + p_g(x)} dx \big) \\ &= \frac{1}{2} \big(\log 4 + L(G, D^*) \big) \end{aligned}$$

Thus,

$$L(G, D^*) = 2D_{JS}(p_r || p_g) - 2 \log 2$$

Essentially the loss function of GAN quantifies the similarity between the generative data distribution p_g and the real sample distribution p_r by JS divergence when the discriminator is optimal. The best G^* that replicates the real data distribution leads to the minimum $L(G^*, D^*) = -2 \log 2$ which is aligned with equations above.

Other Variations of GAN: There are many variations of GANs in different contexts or designed for different tasks. For example, for semi-supervised learning, one idea is to update the discriminator to output real class labels, $1, \dots, K-1$, as well as one fake class label K . The generator model aims to trick the discriminator to output a classification label smaller than K .

Tensorflow Implementation: [carpedm20/DCGAN-tensorflow](#)

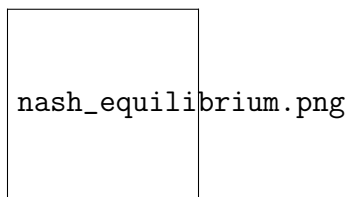


Figure 3: A simulation of our example for updating x to minimize xy and updating y to minimize $-xy$. The learning rate $\eta = 0.1$. With more iterations, the oscillation grows more and more unstable.

4 Problems in GANs#

Although GAN has shown great success in the realistic image generation, the training is not easy; The process is known to be slow and unstable.

4.1 Hard to achieve Nash equilibrium#

Salimans et al. (2016) discussed the problem with GAN’s gradient-descent-based training procedure. Two models are trained simultaneously to find a [Nash equilibrium](#) to a two-player non-cooperative game. However, each model updates its cost independently with no respect to another player in the game. Updating the gradient of both models concurrently cannot guarantee a convergence.

Let’s check out a simple example to better understand why it is difficult to find a Nash equilibrium in an non-cooperative game. Suppose one player takes control of x to minimize $f_1(x) = xy$, while at the same time the other player constantly updates y to minimize $f_2(y) = -xy$.

Because $\frac{\partial f_1}{\partial x} = y$ and $\frac{\partial f_2}{\partial y} = -x$, we update x with $x - \eta \cdot y$ and y with $y + \eta \cdot x$ simultaneously in one iteration, where η is the learning rate. Once x and y have different signs, every following gradient update causes huge oscillation and the instability gets worse in time, as shown in

4.2 Low dimensional supports#

@@ Term Explanation

Manifold A topological space that locally resembles Euclidean space near each point. Precisely, when this Euclidean space is of **dimension n** , the manifold is referred as **n -manifold**.

Support A real-valued function f is the subset of the domain containing those elements which are not mapped to **zero**.

Arjovsky and Bottou (2017) discussed the problem of the [supports](#) of p_r and p_g lying on low dimensional [manifolds](#) and how it contributes to the instability of GAN training thoroughly in a very theoretical paper “[Towards principled methods for training generative adversarial networks](#)”.

The dimensions of many real-world datasets, as represented by p_r , only appear to be **artificially high**. They have been found to concentrate in a lower dimensional manifold. This is actually the fundamental assumption for [Manifold Learning](#). Thinking

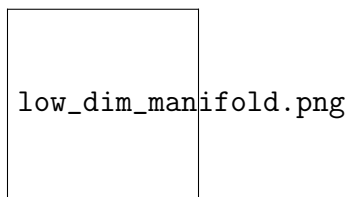


Figure 4: Low dimensional manifolds in high dimension space can hardly have overlaps. (Left) Two lines in a three-dimension space. (Right) Two surfaces in a three-dimension space.

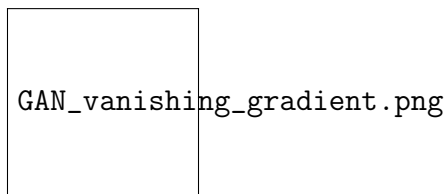


Figure 5: First, a DCGAN is trained for 1, 10 and 25 epochs. Then, with the **generator fixed**, a discriminator is trained from scratch and measure the gradients with the original cost function. We see the gradient norms **decay quickly** (in log scale), in the best case 5 orders of magnitude after 4000 discriminator iterations. (Image source: [Arjovsky and Bottou, 2017](#))

of the real world images, once the theme or the contained object is fixed, the images have a lot of restrictions to follow, i.e., a dog should have two ears and a tail, and a skyscraper should have a straight and tall body, etc. These restrictions keep images away from the possibility of having a high-dimensional free form.

p_g lies in a low dimensional manifolds, too. Whenever the generator is asked to a much larger image like 64x64 given a small dimension, such as 100, noise variable input z , the distribution of colors over these 4096 pixels has been defined by the small 100-dimension random number vector and can hardly fill up the whole high dimensional space.

Because both p_g and p_r rest in low dimensional manifolds, they are almost certainly gonna be disjoint (See Fig. 4). When they have disjoint supports, we are always capable of finding a perfect discriminator that separates real and fake samples 100% correctly. Check the [paper](#) if you are curious about the proof.

4.3 Vanishing gradient#

When the discriminator is perfect, we are guaranteed with $D(x) = 1, \forall x \in p_r$ and $D(x) = 0, \forall x \in p_g$. Therefore the loss function L falls to zero and we end up with no gradient to update the loss during learning iterations. Fig. 5 demonstrates an experiment when the discriminator gets better, the gradient vanishes fast.

As a result, training a GAN faces a **dilemma**:

- If the discriminator behaves badly, the generator does not have accurate feedback and the loss function cannot represent the reality.
- If the discriminator does a great job, the gradient of the loss function drops down to close to zero and the learning becomes super slow or even jammed.

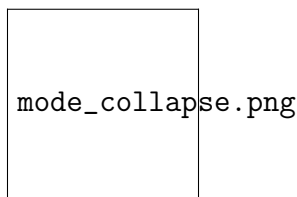


Figure 6: A DCGAN model is trained with an MLP network with 4 layers, 512 units and ReLU activation function, configured to lack a strong inductive bias for image generation. The results shows a significant degree of mode collapse. (Image source: [Arjovsky, Chintala, & Bottou, 2017.](#))

This dilemma clearly is capable to make the GAN training very tough.

4.4 Mode collapse#

During the training, the generator may collapse to a setting where it always produces same outputs. This is a common failure case for GANs, commonly referred to as **Mode Collapse**. Even though the generator might be able to trick the corresponding discriminator, it fails to learn to represent the complex real-world data distribution and gets stuck in a small space with extremely low variety.

4.5 Lack of a proper evaluation metric#

Generative adversarial networks are not born with a good objection function that can inform us the training progress. Without a good evaluation metric, it is like working in the dark. No good sign to tell when to stop; No good indicator to compare the performance of multiple models.

5 Improved GAN Training#

The following suggestions are proposed to help stabilize and improve the training of GANs.

First five methods are practical techniques to achieve faster convergence of GAN training, proposed in “[Improve Techniques for Training GANs](#)”. The last two are proposed in “[Towards principled methods for training generative adversarial networks](#)” to solve the problem of disjoint distributions.

(1) Feature Matching

Feature matching suggests to optimize the discriminator to inspect whether the generator’s output matches expected statistics of the real samples. In such a scenario, the new loss function is defined as $\mathbb{E}_{x \sim p_r} f(x) - \mathbb{E}_{z \sim p_z} f(G(z))$, where $f(x)$ can be any computation of statistics of features, such as mean or median.

(2) Minibatch Discrimination

With minibatch discrimination, the discriminator is able to digest the relationship between training data points in one batch, instead of processing each point independently.

In one minibatch, we approximate the closeness between every pair of samples, $c(x_i, x_j)$, and get the overall summary of one data point by summing up how close it is

to other samples in the same batch, $\phi(x_i) = \sum_j c(x_i, x_j)$. Then $\phi(x_i)$ is explicitly added to the input of the model.

(3) Historical Averaging

For both models, add $\frac{1}{t} \sum_{i=1}^t \|\Theta_i\|^2$ into the loss function, where Θ is the model parameter and Θ_i is how the parameter is configured at the past training time i . This addition piece penalizes the training speed when Θ is changing too dramatically in time.

(4) One-sided Label Smoothing

When feeding the discriminator, instead of providing 1 and 0 labels, use soften values such as 0.9 and 0.1. It is shown to reduce the networks' vulnerability.

(5) Virtual Batch Normalization (VBN)

Each data sample is normalized based on a fixed batch (*reference batch*) of data rather than within its minibatch. The reference batch is chosen once at the beginning and stays the same through the training.

Theano Implementation: [openai/improved-gan](https://github.com/openai/improved-gan)

(6) Adding Noises.

Based on the discussion in the [previous section](#), we now know p_r and p_g are disjoint in a high dimensional space and it causes the problem of vanishing gradient. To artificially “spread out” the distribution and to create higher chances for two probability distributions to have overlaps, one solution is to add continuous noises onto the inputs of the discriminator D .

(7) Use Better Metric of Distribution Similarity

The loss function of the vanilla GAN measures the JS divergence between the distributions of p_r and p_g . This metric fails to provide a meaningful value when two distributions are disjoint.

[Wasserstein metric](#) is proposed to replace JS divergence because it has a much smoother value space. See more in the next section.

6 Wasserstein GAN (WGAN)

6.1 What is Wasserstein distance?

[Wasserstein Distance](#) is a measure of the distance between two probability distributions. It is also called **Earth Mover's distance**, short for EM distance, because informally it can be interpreted as the minimum energy cost of moving and transforming a pile of dirt in the shape of one probability distribution to the shape of the other distribution. The cost is quantified by: the amount of dirt moved $\times$ the moving distance.

Let us first look at a simple case where the probability domain is *discrete*. For example, suppose we have two distributions P and Q , each has four piles of dirt and both have ten shovelfuls of dirt in total. The numbers of shovelfuls in each dirt pile are assigned as follows:

$$\begin{aligned} & P_1 = 3, P_2 = 2, P_3 = 1, P_4 = 4 \quad \& \quad Q_1 = 1, Q_2 = 2, \\ & Q_3 = 4, Q_4 = 3 \end{aligned}$$

In order to change P to look like Q , as illustrated in Fig. 7, we:

- First move 2 shovelfuls from P_1 to $P_2 \Rightarrow (P_1, Q_1)$ match up.
- Then move 2 shovelfuls from P_2 to $P_3 \Rightarrow (P_2, Q_2)$ match up.

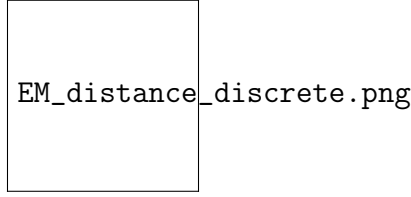


Figure 7: Step-by-step plan of moving dirt between piles in P and Q to make them match.

- Finally move 1 shovelfuls from Q_3 to $Q_4 \Rightarrow (P_3, Q_3)$ and (P_4, Q_4) match up.

If we label the cost to pay to make P_i and Q_i match as δ_i , we would have $\delta_{i+1} = \delta_i + P_i - Q_i$ and in the example:

$$\begin{aligned} \delta_0 &= 0 \\ \delta_1 &= 0 + 3 - 1 = 2 \\ \delta_2 &= 2 + 2 - 2 = 2 \\ \delta_3 &= 2 + 1 - 4 = -1 \\ \delta_4 &= -1 + 4 - 3 = 0 \end{aligned}$$

Finally the Earth Mover's distance is $W = \sum |\delta_i| = 5$.

When dealing with the continuous probability domain, the distance formula becomes:

$$W(p_r, p_g) = \inf_{\gamma \sim \Pi(p_r, p_g)} \int \mathbb{E}_{(x, y) \sim \gamma} |x - y|$$

In the formula above, $\Pi(p_r, p_g)$ is the set of all possible joint probability distributions between p_r and p_g . One joint distribution $\gamma \in \Pi(p_r, p_g)$ describes one dirt transport plan, same as the discrete example above, but in the continuous probability space. Precisely $\gamma(x, y)$ states the percentage of dirt should be transported from point x to y so as to make x follows the same probability distribution of y . That's why the marginal distribution over x adds up to p_g , $\sum_x \gamma(x, y) = p_g(y)$ (Once we finish moving the planned amount of dirt from every possible x to the target y , we end up with exactly what y has according to p_g .) and vice versa $\sum_y \gamma(x, y) = p_r(x)$.

When treating x as the starting point and y as the destination, the total amount of dirt moved is $\gamma(x, y)$ and the travelling distance is $|x - y|$ and thus the cost is $\gamma(x, y) \cdot |x - y|$. The expected cost averaged across all the (x, y) pairs can be easily computed as:

$$\sum_{x, y} \gamma(x, y) |x - y| = \mathbb{E}_{(x, y) \sim \gamma} |x - y|$$

Finally, we take the minimum one among the costs of all dirt moving solutions as the EM distance. In the definition of Wasserstein distance, the $\inf$ (infimum, also known as *greatest lower bound*) indicates that we are only interested in the smallest cost.

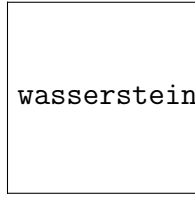
6.2 Why Wasserstein is better than JS or KL divergence?#

Even when two distributions are located in lower dimensional manifolds without overlaps, Wasserstein distance can still provide a meaningful and smooth representation of the distance in-between.

The WGAN paper exemplified the idea with a simple example.

Suppose we have two probability distributions, P and Q :

$$\text{for all } (x, y) \in P, x = 0 \text{ and } y \sim U(0, 1) \\ \text{for all } (x, y) \in Q, x = \theta, 0 \leq \theta \leq 1 \text{ and } y \sim U(0, 1)$$



wasserstein_simple_example.png

Figure 8: There is no overlap between P and Q when $\theta \neq 0$.

When $\theta \neq 0$:

$$\begin{aligned} D_{\text{KL}}(P \parallel Q) &= \sum_{x=0, y \sim U(0, 1)} 1 \cdot \log \frac{1}{0} \\ &= +\infty \\ D_{\text{KL}}(Q \parallel P) &= \sum_{x=\theta, y \sim U(0, 1)} 1 \cdot \log \frac{1}{0} \\ &= +\infty \\ D_{\text{JS}}(P, Q) &= \frac{1}{2} \left(\sum_{x=0, y \sim U(0, 1)} 1 \cdot \log \frac{1}{1/2} \right. \\ &\quad \left. + \sum_{x=\theta, y \sim U(0, 1)} 1 \cdot \log \frac{1}{1/2} \right) = \log 2 \\ W(P, Q) &= \theta \end{aligned}$$

But when $\theta = 0$, two distributions are fully overlapped:

$$\begin{aligned} D_{\text{KL}}(P \parallel Q) &= D_{\text{KL}}(Q \parallel P) = D_{\text{JS}}(P, Q) = 0 \\ W(P, Q) &= 0 \end{aligned}$$

D_{KL} gives us infinity when two distributions are disjoint. The value of D_{JS} has sudden jump, not differentiable at $\theta = 0$. Only Wasserstein metric provides a smooth measure, which is super helpful for a stable learning process using gradient descents.

6.3 Use Wasserstein distance as GAN loss function#

It is intractable to exhaust all the possible joint distributions in $\Pi(p_r, p_g)$ to compute $\inf_{\gamma \sim \Pi(p_r, p_g)}$. Thus the authors proposed a smart transformation of the formula based on the Kantorovich-Rubinstein duality to:

$$W(p_r, p_g) = \frac{1}{K} \sup_{\|f\|_L \leq K} \mathbb{E}_{x \sim p_r}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)]$$

where $\sup$ ([supremum](#)) is the opposite of $\inf$ (infimum); we want to measure the least upper bound or, in even simpler words, the maximum value.

Lipschitz continuity?

The function f in the new form of Wasserstein metric is demanded to satisfy $\|f\|_L \leq K$, meaning it should be [K-Lipschitz continuous](#).

A real-valued function $f: \mathbb{R} \rightarrow \mathbb{R}$ is called K -Lipschitz continuous if there exists a real constant $K \geq 0$ such that, for all $x_1, x_2 \in \mathbb{R}$,

$$|f(x_1) - f(x_2)| \leq K |x_1 - x_2|$$

Here K is known as a Lipschitz constant for function $f(\cdot)$. Functions that are everywhere continuously differentiable is Lipschitz continuous, because the derivative, estimated as $\frac{|f(x_1) - f(x_2)|}{|x_1 - x_2|}$, has bounds. However, a Lipschitz continuous function may not be everywhere differentiable, such as $f(x) = |x|$.

Explaining how the transformation happens on the Wasserstein distance formula is worthy of a long post by itself, so I skip the details here. If you are interested in how to compute Wasserstein metric using linear programming, or how to transfer Wasserstein metric into its dual form according to the Kantorovich-Rubinstein Duality, read this [awesome post](#).

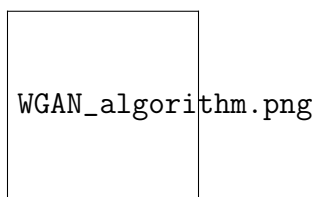


Figure 9: Algorithm of Wasserstein generative adversarial network. (Image source: [Arjovsky, Chintala, & Bottou, 2017](#).)

Suppose this function f_w comes from a family of K -Lipschitz continuous functions, $\{f_w\}_{w \in W}$, parameterized by w . In the modified Wasserstein-GAN, the “discriminator” model is used to learn w to find a good f_w and the loss function is configured as measuring the Wasserstein distance between p_r and p_g .

$$L(p_r, p_g) = W(p_r, p_g) = \max_{w \in W} \mathbb{E}_{x \sim p_r}[f_w(x)] - \mathbb{E}_{z \sim p_r(z)}[f_w(g_\theta(z))]$$

Thus the “discriminator” is not a direct critic of telling the fake samples apart from the real ones anymore. Instead, it is trained to learn a K -Lipschitz continuous function to help compute Wasserstein distance. As the loss function decreases in the training, the Wasserstein distance gets smaller and the generator model’s output grows closer to the real data distribution.

One big problem is to maintain the K -Lipschitz continuity of f_w during the training in order to make everything work out. The paper presents a simple but very practical trick: After every gradient update, clamp the weights w to a small window, such as $[-0.01, 0.01]$, resulting in a compact parameter space W and thus f_w obtains its lower and upper bounds to preserve the Lipschitz continuity.

Compared to the original GAN algorithm, the WGAN undertakes the following changes:

- After every gradient update on the critic function, clamp the weights to a small fixed range, $[-c, c]$.
- Use a new loss function derived from the Wasserstein distance, no logarithm anymore. The “discriminator” model does not play as a direct critic but a helper for estimating the Wasserstein metric between real and generated data distribution.
- Empirically the authors recommended [RMSProp](#) optimizer on the critic, rather than a momentum based optimizer such as [Adam](#) which could cause instability in the model training. I haven’t seen clear theoretical explanation on this point through.

Sadly, Wasserstein GAN is not perfect. Even the authors of the original WGAN paper mentioned that “*Weight clipping is a clearly terrible way to enforce a Lipschitz constraint*” (Oops!). WGAN still suffers from unstable training, slow convergence after weight clipping (when clipping window is too large), and vanishing gradients (when clipping window is too small).

Some improvement, precisely replacing weight clipping with **gradient penalty**, has been discussed in [Gulrajani et al. 2017](#). I will leave this to a future post.

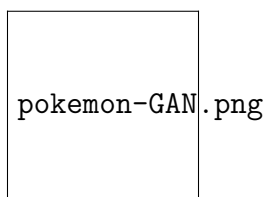


Figure 10: Train [carpedm20/DCGAN-tensorflow](#) on a set of Pokemon sprite images. The sample outputs are listed after training epoches = 7, 21, 49.

7 Example: Create New Pokemons!#

Just for fun, I tried out [carpedm20/DCGAN-tensorflow](#) on a tiny dataset, [Pokemon sprites](#). The dataset only has 900-ish pokemon images, including different levels of same pokemon species.

Let's check out what types of new pokemons the model is able to create. Unfortunately due to the tiny training data, the new pokemons only have rough shapes without details. The shapes and colors do look better with more training epoches! Hooray!

If you are interested in a commented version of [carpedm20/DCGAN-tensorflow](#) and how to modify it to train WGAN and WGAN with gradient penalty, check [lilianweng/unified-gan-tensorflow](#).

Cited as:

```
@article{weng2017gan,
  title   = "From GAN to WGAN",
  author  = "Weng, Lilian",
  journal = "lilianweng.github.io",
  year    = "2017",
  url     = "https://lilianweng.github.io/posts/2017-08-20-gan/"
}
```

OR

```
@misc{weng2019gan,
  title={From GAN to WGAN},
  author={Lilian Weng},
  year={2019},
  eprint={1904.08994},
  archivePrefix={arXiv},
  primaryClass={cs.LG}
}
```

8 References#

- [1] Goodfellow, Ian, et al. "Generative adversarial nets." NIPS, 2014.
- [2] Tim Salimans, et al. "Improved techniques for training gans." NIPS 2016.

[3] Martin Arjovsky and Léon Bottou. “Towards principled methods for training generative adversarial networks.” arXiv preprint arXiv:1701.04862 (2017).

[4] Martin Arjovsky, Soumith Chintala, and Léon Bottou. “Wasserstein GAN.” arXiv preprint arXiv:1701.07875 (2017).

[5] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, Aaron Courville. Improved training of wasserstein gans. arXiv preprint arXiv:1704.00028 (2017).

[6] Computing the Earth Mover’s Distance under Transformations

[7] Wasserstein GAN and the Kantorovich-Rubinstein Duality

[8] zhuanlan.zhihu.com/p/25071913

[9] Ferenc Huszár. “How (not) to Train your Generative Model: Scheduled Sampling, Likelihood, Adversary?.” arXiv preprint arXiv:1511.05101 (2015).

- [Gan](#)
- [Long-Read](#)
- [Generative-Model](#)
- [Image-Generation](#)
- [Math-Heavy](#)

Anatomize Deep Learning with Information Theory
How to Explain the Prediction of a Machine Learning Model?

© 2025 Lil'Log Powered by Hugo & PaperMod

9 Citation

Please cite this work as:

Weng, Lilian. “[Title](#)”. Lil 'Log (May 2025). <https://lilianweng.github.io/posts/2017-08-20-gan/>

Or use the BibTeX citation:

```
@article{weng2025,
  title = {},
  author = {Weng, Lilian},
  journal = {lilianweng.github.io},
  year = {2025}, month = {May},
  url = {}
}
```

10 References

Tags: [tag1](#) [tag2](#)

[⟨ Title Title ⟩](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)